JAVA COURSE

{ debugging }



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.1

INDICE

01 I tipi di errore

02 Le tecniche di debugging

03 Debugging in pratica

01
I TIPI DI ERRORE



TRE CATEGORIE

Ogni programma può contenere errori. Si dividono in tre famiglie, ciascuna con caratteristiche diverse

- ***Errori di compilazione:*** *il compilatore li trova prima che il programma parta*
- ***Errori di runtime:*** *il programma parte ma si interrompe durante l'esecuzione*
- ***Errori logici:*** *il programma funziona ma produce un risultato sbagliato*
- ***Difficoltà:*** *cresce dall'alto verso il basso, gli errori logici sono i più insidiosi*

ERRORI DI COMPILAZIONE

Il compilatore `javac` analizza il codice prima di eseguirlo. Se trova un errore di sintassi o di tipo, **non genera il file** `.class` e mostra un messaggio

- ***Prima la sintassi:*** `javac` *segnala il* `;` *mancante alla riga 4 e si ferma*
- ***Poi i tipi:*** *corretto il* `;` *, alla compilazione successiva compare* `incompatible types` *alla riga 3*

```java
public class Esempio {
    public static void main(String[] args) {
        int x = "ciao";          // tipo incompatibile
        System.out.println(x)    // manca il ;
    }
}
```

Terminale

```
$ javac Esempio.java
Esempio.java:4: error: ';' expected
        System.out.println(x)
                           ^
1 error
```

MESSAGGI COMUNI DEL COMPILATORE

I messaggi più frequenti e il loro significato

Messaggio	Causa tipica
`';' expected`	Manca il punto e virgola a fine istruzione
`cannot find symbol`	Variabile non dichiarata o nome scritto male
`incompatible types`	Tipo sbagliato nell'assegnazione
`unclosed string literal`	Mancano le virgolette di chiusura
`class Saluto is public, should be declared in a file named Saluto.java`	Il nome della classe pubblica non corrisponde al nome del file

ERRORI DI RUNTIME

Il programma compila senza problemi ma si interrompe durante l'esecuzione. Java lancia un'**eccezione** e stampa uno **stack trace**

```java
public class Esempio {
    public static void main(String[] args) {
        int[] numeri = {1, 2, 3};
        System.out.println(numeri[5]);   // indice fuori dai limiti
    }
}
```

Terminale

```
$ java Esempio.java
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3
        at Esempio.main(Esempio.java:4)
```

ECCEZIONI COMUNI

Le eccezioni di runtime che si incontrano più spesso nei primi programmi

Eccezione	Quando si verifica
`ArithmeticException`	Divisione intera per zero
`ArrayIndexOutOfBoundsException`	Indice fuori dai limiti dell'array
`NullPointerException`	Si chiama un metodo su una variabile che vale `null`
`InputMismatchException`	`Scanner` legge un tipo diverso da quello atteso
`StringIndexOutOfBoundsException`	Indice fuori dai limiti di una stringa

LO STACK TRACE

Lo stack trace si legge **dall'alto verso il basso**: prima cosa è successo, poi dove, poi chi ha chiamato quel codice

- ***Riga 1:*** *il tipo di eccezione e il messaggio*
- ***Riga 2:*** *dove è avvenuto l'errore, il metodo* `dividi` *alla riga 8 di* `Calcolo.java`*: si parte da qui*
- ***Riga 3:*** *chi ha chiamato quel metodo,* `main` *alla riga 3*

Terminale

```
$ java Calcolo
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at Calcolo.dividi(Calcolo.java:8)
        at Calcolo.main(Calcolo.java:3)
```

ERRORI LOGICI

Il programma compila, esegue senza eccezioni, ma il risultato è **sbagliato**. Il compilatore e il runtime non possono aiutare: solo il programmatore conosce il risultato atteso

- *Nessun messaggio di errore, nessuna eccezione*
- *Servono **test** e **debugging** per trovarli*

```
// dovrebbe calcolare l'area del rettangolo
int base = 5;
int altezza = 10;
int area = base + altezza;   // errore: + invece di *
System.out.println(area);    // stampa 15 invece di 50
```

CHE TIPO DI ERRORE?

Per ogni frammento, indica se l'errore è di compilazione, di runtime o logico

```
// A
int x = 10
System.out.println(x);

// B
int risultato = 100 / 0;

// C
double media = (80 + 90) / 2;   // risultato atteso: 85.0
System.out.println(media);      // stampa 85.0? o no?

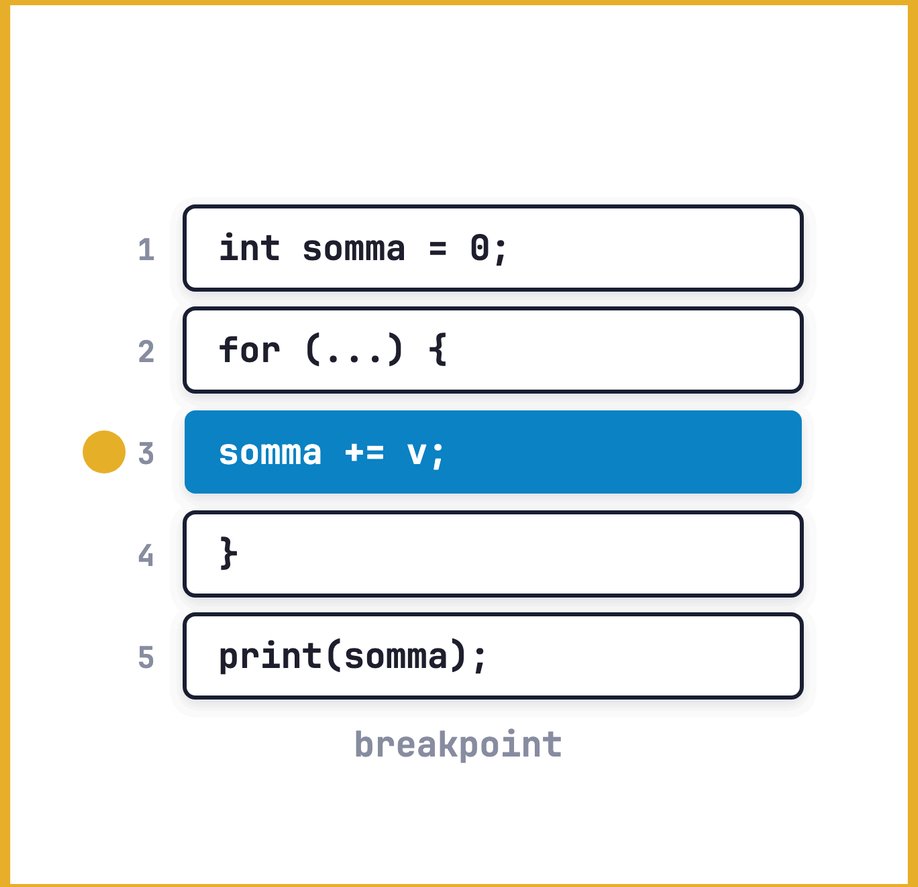
// D
String nome = null;
System.out.println(nome.length());
```

RISPOSTE

- ***A, compilazione:*** *manca il punto e virgola dopo* `int x = 10`
- ***B, runtime:*** *la divisione intera per zero lancia* `ArithmeticException`
- ***C, logico:*** `(80 + 90) / 2` *vale* `85` *(divisione intera), quindi il risultato è corretto solo per caso. Con* `(81 + 90) / 2` *stamperebbe* `85.0` *invece di* `85.5`
- ***D, runtime:*** `nome` *vale* `null`*, chiamare* `length()` *lancia* `NullPointerException`

02

LE TECNICHE DI DEBUGGING



CHE COS'È IL DEBUGGING

Il **debugging** è il processo di trovare e correggere gli errori (*bug*) in un programma. Non è una scienza esatta: servono metodo, pazienza e pratica

- ***Origine della parola:*** *già Thomas Edison, nel 1878, chiamava bug i difetti delle sue invenzioni*
- ***Aneddoto famoso:*** *nel 1947 il gruppo di Grace Hopper trovò una vera falena nel computer Harvard Mark II e la incollò nel registro di lavoro*
- ***Tempo:*** *è una delle attività su cui i programmatori spendono più tempo*
- ***Bravura:*** *un buon programmatore non è chi non fa errori, ma chi li trova in fretta*

PRINTLN STRATEGICI

La tecnica più semplice: aggiungere `System.out.println` per stampare i valori delle variabili nei punti sospetti

- ***Prefisso*** `DEBUG`***:*** *rende le stampe facili da trovare e da rimuovere*
- ***Nome e valore:*** *stampare sempre entrambi*

```java
int somma = 0;
for (int i = 1; i <= 5; i++) {
    somma += i;
    System.out.println("DEBUG i=" + i + " somma=" + somma);
}
System.out.println("Totale: " + somma);
```

Terminale

```
DEBUG i=1 somma=1
DEBUG i=2 somma=3
DEBUG i=3 somma=6
DEBUG i=4 somma=10
DEBUG i=5 somma=15
Totale: 15
```

IL DEBUGGER DELL'IDE

L'IDE offre un debugger grafico: un modo per eseguire il programma passo dopo passo e osservare le variabili senza modificare il codice

- ***Breakpoint:*** *un punto in cui l'esecuzione si ferma*
- ***Step Over:*** *esegue la riga corrente e passa alla successiva*
- ***Step Into:*** *entra dentro il metodo chiamato nella riga corrente*
- ***Step Out:*** *esce dal metodo corrente e torna al chiamante*
- ***Watch:*** *mostra il valore di una variabile o espressione in tempo reale*
- ***Resume:*** *riprende l'esecuzione fino al prossimo breakpoint*

IL FLUSSO DEL DEBUGGER

I passaggi per usare il debugger in IntelliJ o VS Code

- ***1.*** *Cliccare accanto al numero di riga per mettere un* ***breakpoint*** *(pallino rosso)*
- ***2.*** *Avviare il programma in modalità* ***Debug*** *(icona dell'insetto)*
- ***3.*** *L'esecuzione si ferma al breakpoint:* ***ispezionare le variabili*** *nel pannello del debugger*
- ***4.*** *Usare* ***Step Over*** *per avanzare riga per riga*
- ***5.*** *Confrontare il valore* ***atteso*** *con quello* ***effettivo***
- ***6.*** *La prima riga che produce un valore sbagliato è quella del bug*

ISOLARE IL PROBLEMA

Quando il programma è lungo, non serve cercare l'errore ovunque. Si divide il codice in parti e si controlla ogni parte separatamente

- ***Commentare:*** *escludere temporaneamente blocchi di codice*
- ***Testare:*** *provare un singolo metodo con valori noti*
- ***Dimezzare:*** *il bug è prima o dopo la metà? Si procede per esclusione*
- ***Ridurre:*** *più il pezzo di codice isolato è piccolo, più l'errore è facile da trovare*

LEGGERE IL MESSAGGIO DI ERRORE

Gran parte della soluzione è nel messaggio di errore: dice cosa è andato storto e dove. Leggerlo è sempre il primo passo

- ***Prima riga:*** *il file, la riga e il tipo di errore*
- ***Freccia*** `^` ***:*** *il punto esatto nella riga*
- ***Righe sotto:*** *i dettagli, qui i tipi dei due operandi di* `+`*, un* `int` *e un array*

Terminale

```
$ javac Calcolo.java
Calcolo.java:4: error: bad operand types for binary operator '+'
        int totale = 10 + voti;
                         ^
  first type:  int
  second type: int[]
1 error
```

RUBBER DUCK DEBUGGING

Una tecnica sorprendentemente efficace: **spiegare il codice ad alta voce**, riga per riga, come se si parlasse a una paperella di gomma

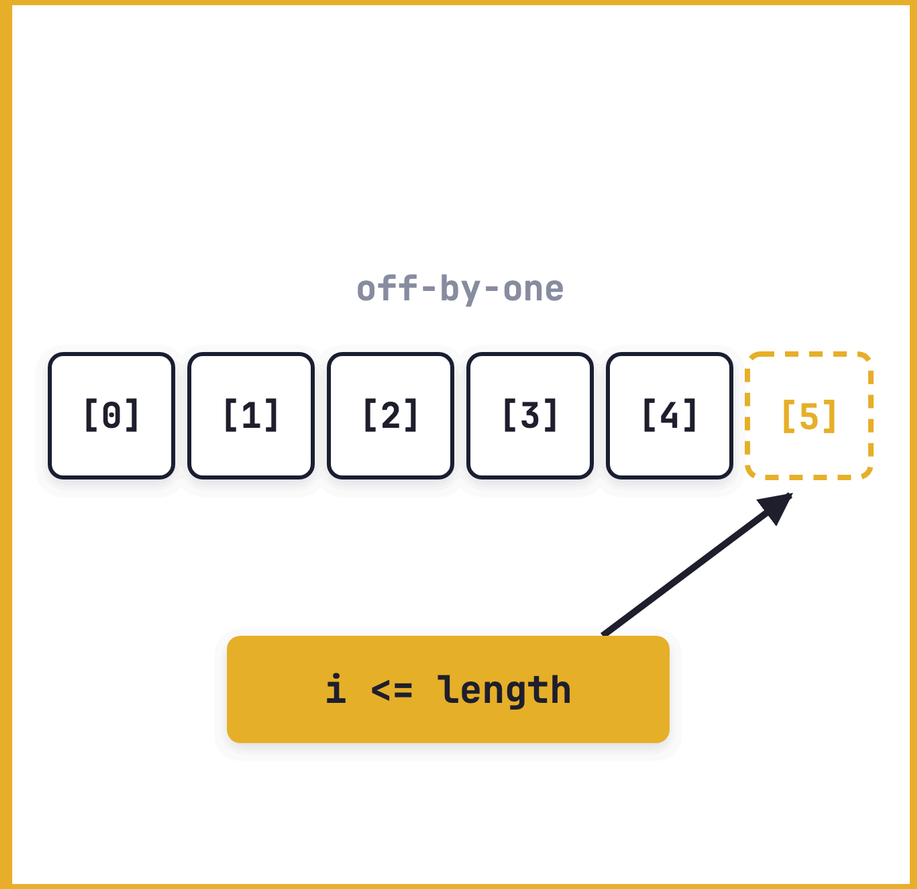
- *Non serve un interlocutore esperto: basta verbalizzare il ragionamento*
- *Quando si spiega "questa riga dovrebbe fare X", spesso ci si accorge che in realtà fa Y*
- *Funziona perché costringe a rallentare e a leggere ogni riga con attenzione*
- *Molti programmatori professionisti la usano davvero*

CHECKLIST DI DEBUGGING

Una sequenza da seguire quando si incontra un errore

- ***1.*** *Leggere il messaggio di errore con attenzione*
- ***2.*** *Andare alla riga indicata nel messaggio*
- ***3.*** *Se la riga sembra corretta, controllare anche quelle precedenti*
- ***4.*** *Aggiungere* `println` *o usare il debugger per ispezionare i valori*
- ***5.*** *Confrontare il valore effettivo con quello atteso*
- ***6.*** *Correggere, ricompilare, testare di nuovo*

03
DEBUGGING IN PRATICA



OFF-BY-ONE NEI CICLI

L'errore più comune nei cicli: il contatore parte da un valore sbagliato o la condizione include un elemento di troppo (o uno di meno)

- *Controllare sempre che il primo e l'ultimo valore siano quelli attesi*

```
// dovrebbe stampare i numeri da 1 a 5
for (int i = 0; i <= 5; i++) {
    System.out.println(i);
}
// stampa 0, 1, 2, 3, 4, 5 → 6 numeri invece di 5

// versione corretta
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

NULLPOINTEREXCEPTION

Si verifica quando si chiama un metodo su una variabile che vale `null`. È come cercare di aprire una porta che non esiste

- ***Cause comuni:*** *variabile rimasta a* `null`*, metodo che restituisce* `null`
- ***Dove guardare:*** *lo stack trace indica la riga esatta*

```
String nome = null;
int lunghezza = nome.length();  // NullPointerException

// prevenzione: controllare prima di usare
if (nome != null) {
    System.out.println(nome.length());
}
```

DIVISIONE INTERA PER SBAGLIO

Quando si dividono due `int`, Java tronca il risultato. Se il risultato atteso è decimale, il programma produce un valore sbagliato senza errori

- *È un errore logico: nessuna eccezione, nessun messaggio*
- *Il cast* `(double)` *converte l'operando prima della divisione*

```
int totale = 7;
int quantita = 2;
double media = totale / quantita;
System.out.println(media);      // stampa 3.0 invece di 3.5

// soluzione: forzare la divisione double
double media2 = (double) totale / quantita;
System.out.println(media2);     // stampa 3.5
```

ERRORI COMUNI

I bug più frequenti nei primi programmi Java

Errore	Causa	Correzione
`=` invece di `==`	Assegnazione al posto del confronto	Usare `==` nelle condizioni
`==` con le stringhe	Confronta i riferimenti, non il contenuto	Usare `.equals()`
Off-by-one nel ciclo	`<` al posto di `<=`, indice di partenza sbagliato	Verificare primo e ultimo valore
Divisione intera	`int / int` tronca il decimale	Cast a `(double)`
Variabile locale non inizializzata	Errore di compilazione: `variable x might not have been initialized`	Assegnarle un valore prima di usarla
Parentesi graffe mancanti	Solo la prima istruzione appartiene al blocco	Usare sempre le graffe

CONFRONTO CON PYTHON

Errori simili, messaggi diversi. Il concetto di fondo è lo stesso: leggere il messaggio e trovare la riga

Situazione	Python	Java
Tipo sbagliato	`TypeError`	`incompatible types` (compilazione)
Indice fuori limiti	`IndexError`	`ArrayIndexOutOfBoundsException`
Divisione per zero	`ZeroDivisionError`	`ArithmeticException`
Variabile inesistente	`NameError`	`cannot find symbol` (compilazione)
Traccia dell'errore	Traceback	Stack trace

TROVA L'ERRORE

Il programma dovrebbe stampare i numeri pari da 2 a 10. Cosa stampa in realtà? Che tipo di errore è?

```
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 1) {
        System.out.println(i);
    }
}
// ??
```

TROVA L'ERRORE

Il programma dovrebbe calcolare la media di tre voti. Qual è il risultato effettivo?

```
int voto1 = 28;
int voto2 = 30;
int voto3 = 25;
double media = voto1 + voto2 + voto3 / 3;
System.out.println("Media: " + media);
// ??
```

RISPOSTE AI QUIZ

- ***Quiz 1, errore logico:*** *stampa 1, 3, 5, 7, 9, perché* `i % 2 == 1` *seleziona i numeri* ***dispari****. La condizione corretta è* `i % 2 == 0`

- ***Quiz 2, errore logico:*** *la divisione ha precedenza sulla somma, quindi* `voto3 / 3` *viene calcolato prima (25 / 3 = 8, divisione intera). Stampa* `Media: 66.0` *invece di circa 27.67. Servono le parentesi:* `(voto1 + voto2 + voto3) / 3.0`

DOMANDE?



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.1